

Nom:

Prénom:

Note:

Exercice 1 ▶

/3 points

Compréhension et Vocabulaire : Un développeur utilise un dictionnaire pour stocker l'inventaire d'un joueur :

</> Code Python

```
1 inventaire = {"potions": 5, "pieces": 150, "epee": 1}
```

1. Que renvoie la commande `type(inventaire)`?
2. Dans le couple `"pieces": 150`, identifiez la clé et la valeur.
3. Quelle instruction permet d'afficher le nombre de `"potions"`?
4. Le joueur achète une potion supplémentaire. Écrivez l'instruction qui met à jour la quantité de potions à 6.
5. Le joueur ramasse un nouvel objet : `"bouclier"` (quantité 1). Écrivez l'instruction permettant d'ajouter cet élément au dictionnaire.

Exercice 2 ▶

/2 points

On considère un dictionnaire de produits et leurs prix en euros :

</> Code Python

```
1 catalogue = {"Clavier": 45, "Souris": 25, "Ecran": 180, "Tapis": 15}
```

1. Ecrire un script Python qui affiche le nom et le prix de chaque produit du catalogue.
L'affichage doit être au format : `Produit: Prix euros` (exemple : `Clavier: 45 euros`).

2. Compléter le script Python pour qu'il affiche uniquement les **noms** des produits dont le prix est inférieur à 30 euros.

</> Code Python

```
1 for ..... , ..... in catalogue. ....:
2     if ..... :
3         print(produit)
```

Exercice 3 ►

/1 point

Compléter cet algorithme qui permet de compter les occurrences de chaque caractère dans une chaîne.

Code Python

```

1 texte = "abracadabra"
2 frequences = {}
3
4 for caractere in texte:
5     if caractere in .....:
6         frequences[caractere] = .....
7     else:
8         frequences[caractere] = .....
9
10 print(frequences) # Doit afficher {'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}

```

Exercice 4 ►

/2 points

Structures Complexes : Bibliothèque : On utilise une liste de dictionnaires pour gérer des livres :

Code Python

```

1 biblio = [
2     {"titre": "1984", "auteur": "Orwell", "disponible": True},
3     {"titre": "Dune", "auteur": "Herbert", "disponible": False},
4     {"titre": "Fondation", "auteur": "Asimov", "disponible": True} ]

```

1. Quelle instruction permet d'accéder au dictionnaire du livre "Dune" ?
2. Que renvoie l'instruction `biblio[2]["auteur"]` ?
3. **Écriture de fonction :** Complétez la fonction ci-dessous qui renvoie `True` si un livre est disponible, et `False` sinon.

Code Python

```

1 def est_dispo(titre_recherche, table_livres):
2     ''' Renvoie True si le livre est disponible, False sinon.
3     titre_recherche : le titre du livre a chercher (str)
4     table_livres : une liste de dictionnaires representant les livres (list)
5     '''
6     for livre in table_livres:
7         if livre["titre"] == .....:
8             return .....["disponible"]
9     return "Livre non reference"

```

Exercice 5 ▶

/2 points

On s'intéresse aux notes obtenues par des étudiants. Ces notes sont rangées dans un dictionnaire de dictionnaires de ce type :

</> Code Python

```
1 notes = {  
2     "Alice": {"Maths": 15, "NSI": 18, "Histoire": 12},  
3     "Bob": {"Maths": 10, "NSI": 14, "Histoire": 16},  
4     "Charlie": {"Maths": 20, "NSI": 19, "Histoire": 17}  
5 }
```

1. Quelle instruction permet d'afficher la note de Bob en NSI?
2. Écrire une fonction qui calcule la moyenne d'un étudiant donné. La fonction doit prendre en paramètre le nom de l'étudiant et le dictionnaire des notes, et renvoyer la moyenne.

</> Code Python

```
def moyenne(nom_etudiant, table_notes):  
    ''' Renvoie la moyenne de l'etudiant.  
    nom_etudiant : le nom de l'etudiant (str)  
    table_notes : un dictionnaire de dictionnaires representant les notes  
    '''  
  
  
  
assert moyenne("Alice", notes) == 15.0  
assert moyenne("Bob", notes) == 13.34  
assert moyenne("Charlie", notes) == 18.67
```

3. Écrire une fonction qui renvoie le nom de l'étudiant ayant la meilleure moyenne générale. La fonction doit prendre en paramètre le dictionnaire des notes, et renvoyer le nom de l'étudiant avec la meilleure moyenne.

</> Code Python

```
def meilleur_etudiant(table_notes):  
    ''' Renvoie le nom de l'etudiant ayant la meilleure moyenne.  
    table_notes : un dictionnaire de dictionnaires representant les notes  
    '''  
  
  
  
assert meilleur_etudiant(notes) == "Charlie"
```